Football Penalty Challenge

Product Requirements Document (PRD)

Type: 2D arcade-style football penalty game **Platform:** Windows

Technology: Godot 4.7.2 .NET, C#, .NET SDK, Git/GitHub

1. Project Overview

Football Penalty Challenge is a 2D arcade-style football game focused on repeated penalty shots. The player aims a shot, selects shot power, and attempts to score against a goalkeeper. The player starts with 3 lives and a score of 0, then continues taking penalties until all 3 lives are lost.

- **Goal:** score as many points as possible before losing all 3 lives.
- **Goal:** +10 points.
- **Saved or missed shot:** -1 life.
- **Attempts:** no fixed number of penalty attempts.
- **Game end:** lives reach 0.

2. Core Gameplay

- Select an aim direction or target zone.
- Set shot power using the power meter.
- Press Shoot.
- The ball travels toward the selected target.
- The goalkeeper reacts and attempts to save.
- A goal adds 10 points.
- A save or miss removes 1 life.
- If lives remain, another penalty begins.
- When lives reach 0, show the final score and game-over screen.

3. Gameplay Systems

Aim System

The goal is divided into target zones. The player chooses where to aim before shooting. The prototype can use simple selectable zones rather than realistic ball physics.

Shot Power

A shot power meter allows the player to choose the strength of the shot. The selected power is passed to the shot controller and used for ball movement.

Goalkeeper Difficulty

The goalkeeper system uses a base Goalkeeper class and three derived classes. Each difficulty can use different reaction timing or save behaviour while sharing common goalkeeper behaviour.

Class	Purpose
Goalkeeper	Base/abstract class containing common goalkeeper behaviour.
EasyGoalkeeper	Slower or less accurate reactions.
MediumGoalkeeper	Balanced reaction and save behaviour.

Class	Purpose
HardGoalkeeper	Faster or more accurate reactions.

4. User Interface

Screen	Required elements
Main Menu	Game title, Start Game button, basic navigation.
Gameplay	Goal, goalkeeper, ball, aim/target zones, power meter, Shoot button, score, lives.
Game Over	Game Over message, final score, Restart button.

5. Object-Oriented Design

Class	Responsibility
GameManager	Controls game state, starts rounds, and checks game-over conditions.
Player	Represents the player and coordinates player actions.
ShotController	Handles aiming, shot power, and triggering a shot.
Ball	Represents the football and its movement/result.
Goalkeeper	Base/abstract class for common goalkeeper behaviour.
EasyGoalkeeper	Implements easy goalkeeper behaviour.
MediumGoalkeeper	Implements medium goalkeeper behaviour.
HardGoalkeeper	Implements hard goalkeeper behaviour.
ScoreManager	Stores and updates score and lives.
UIManager	Updates menu, gameplay, and game-over UI.
GameResult	Represents outcome/result information.

6. OOP Concepts Demonstrated

- **Encapsulation:** keep game state and behaviour inside appropriate classes, such as ScoreManager managing score and lives.
- **Inheritance:** EasyGoalkeeper, MediumGoalkeeper, and HardGoalkeeper inherit from Goalkeeper.
- **Polymorphism:** gameplay code can use a Goalkeeper reference while each derived class provides its own behaviour.
- **Composition/associations:** GameManager coordinates Player, ScoreManager, UIManager, and other components.
- **Separation of responsibilities:** shooting, scoring, goalkeeper behaviour, gameplay state, and UI are separated.

7. Proposed Project Structure

```
FootballPenaltyChallenge/
■■■ Assets/
■ ■■■ Images/
■ ■■■ Audio/
■ ■■■ Fonts/
■■■ Scenes/
■ ■■■ MainMenu/MainMenu.tscn
■ ■■■ Game/Game.tscn
■ ■■■ UI/GameOver.tscn
■■■ Scripts/
■ ■■■ Core/
■ ■ ■■■ GameManager.cs
■ ■ ■■■ ScoreManager.cs
■ ■■■ Player/
■ ■ ■■■ Player.cs
■ ■ ■■■ ShotController.cs
■ ■■■ Ball/
■ ■ ■■■ Ball.cs
■ ■■■ Goalkeeper/
■ ■ ■■■ Goalkeeper.cs
```

```
■ ■ ■■■ EasyGoalkeeper.cs
■ ■ ■■■ MediumGoalkeeper.cs
■ ■ ■■■ HardGoalkeeper.cs
■ ■■■ UI/
■ ■■■ UIManager.cs
■■■ project.godot
```

8. MVP Requirements

- Main menu and Start Game flow.
- Gameplay scene with goal, goalkeeper, ball, and player controls.
- Three starting lives and score of 0.
- Aim/target selection and shot power meter.
- Shoot action and ball movement.
- Goalkeeper reaction.
- Goal detection and save/miss detection.
- Score increase of 10 for a goal.
- Life reduction of 1 for a save or miss.
- Continuous play while lives remain.
- Game-over screen when lives reach 0.
- Final score display and restart option.
- C# classes structured to demonstrate OOP concepts.

9. Stretch Features

- Selectable goalkeeper difficulty.
- Different shot/target zones.
- Combo or streak scoring.
- Basic statistics such as goals, saves, and shots.
- Sound effects and background audio.
- Simple goalkeeper and ball animations.
- Additional visual polish.

10. Development Phases

Phase	Work
1. Setup	Create Godot .NET project, C# structure, folders, and base scene.
2. Basic Scene	Create goal, ball, goalkeeper, player, and gameplay layout.
3. Shooting	Implement aiming, power meter, Shoot action, and ball movement.
4. Goalkeeper	Implement Goalkeeper base class and difficulty behaviours.
5. Score/Lives	Implement ScoreManager, goals, saves/misses, and game-over state.
6. UI	Add main menu, gameplay HUD, game-over screen, and restart flow.
7. Polish	Improve visuals, feedback, animation, and optional stretch features.

11. Acceptance Criteria

- The game launches successfully on Windows using Godot 4.7.2 .NET.
- The player can start a new game from the main menu.

- A new game begins with 3 lives and 0 score.
- The player can choose an aim direction/target zone and shot power.
- The player can shoot the ball.
- The goalkeeper reacts to the shot.
- A successful goal adds 10 points.
- A saved or missed shot removes 1 life.
- The player can continue taking penalties while at least 1 life remains.
- The game ends when lives reach 0.
- The game-over screen displays the final score.
- The player can restart and begin another game.
- The implementation clearly uses the planned C# classes and OOP concepts.

12. Scope Principle

Build the simplest playable version first. Prioritise a working penalty loop before visual polish or advanced mechanics. Target progression: **working** → **structured** → **playable** → **polished**.

13. Assignment Context

This PRD is based on the SIT771 Object-Oriented Development Distinction Task 6.3: Custom Program Pitch. The task asks for a program of the student's own creation, a short report describing the program, sketches/ideas for the user interface, and ideas for classes, with emphasis on demonstrating course concepts and UML design.